

Drawing AR Quivers of Finite-Dimensional Algebras with GAP

Quick Start

Launch the container

A VPN is required to access the binder when you are in China.

Automatic launch: click , or open one of the following links:

- Stable version: https://mybinder.org/v2/gh/zhangchenchengSJTU/GAP_AR_Quiver/main
- Testing version: https://mybinder.org/v2/gh/zhangchenchengSJTU/GAP_AR_Quiver/tes_t

Manual launch: open [Binder](#) and choose the GitHub repository option. Complete only the following steps.

1. Make sure that the `GitHub repository name or URL` field is set to `GitHub`. Paste `https://github.com/zhangchenchengSJTU/GAP_AR_Quiver` into the box on the right.
2. Click `Launch` directly.
3. Wait until the environment is ready. Binder will then redirect you to the Jupyter Notebook page.

Enter Jupyter Notebook

After entering Jupyter Notebook, the browser address should look like

`https://hub.bids.mybinder.org/user/zhangchenchengsjtu-gap_ar_quiver-???????/tree`. In the root directory, you should see the following three items:

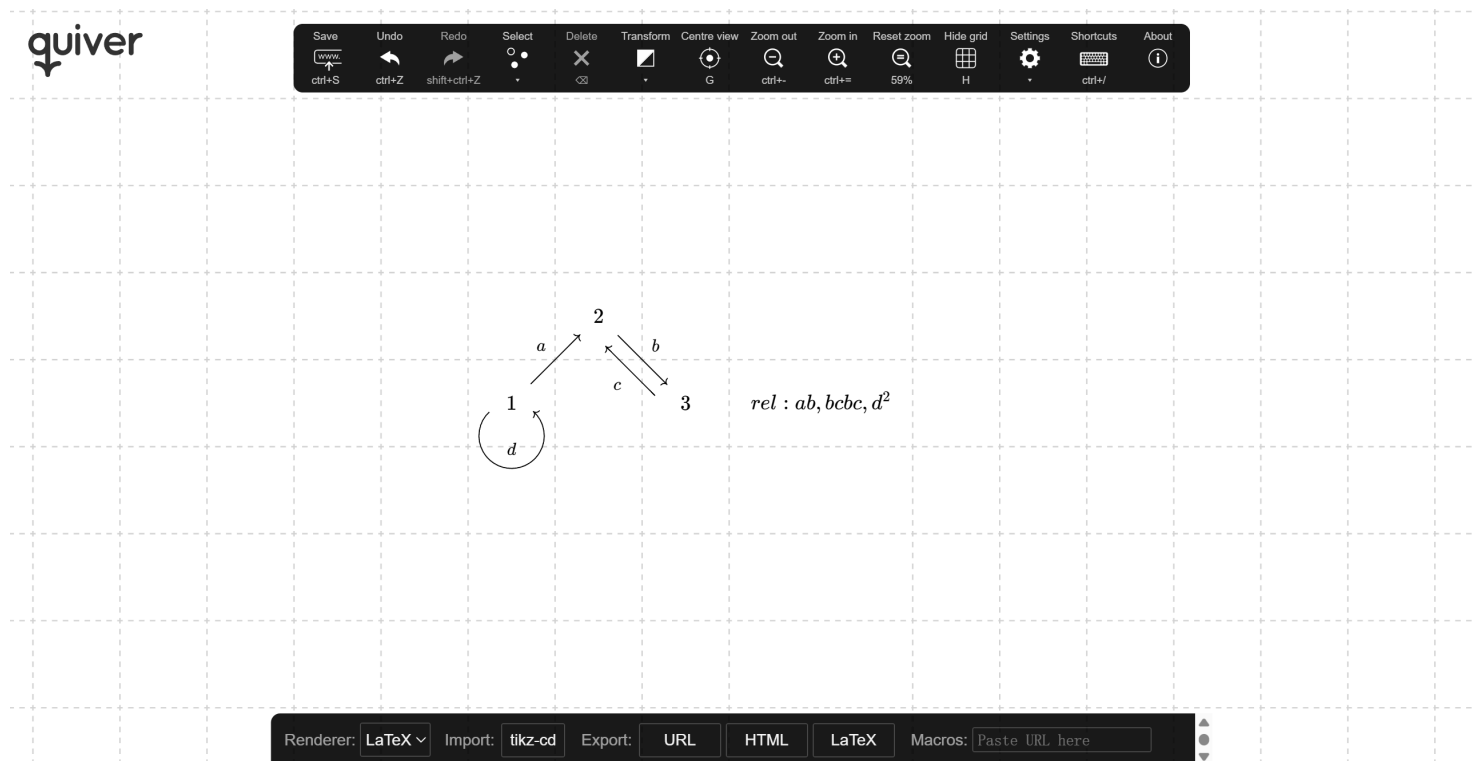
- `Dockerfile`: the environment specification, mainly for developers. Users do not need to edit it.
- `Readme.md`: this documentation file.
- `ARquiver`: the working directory for drawing AR quivers. After entering this folder, you will see:
 - `source`: the source-code directory. Users usually do not need to inspect it.
 - `run.ipynb`: the notebook used to run the computation and rendering scripts.
 - `example.txt`, `untitled.txt`, and similar files: input files containing quiver data.

Draw the path algebra

Use <https://q.uiver.app/> to draw a quiver with relations. Please follow these conventions:

- vertices of the quiver should be positive integers;
- arrows should be simple Latin letters or Greek letters written in $LATEX$ format, such as `a` or `\alpha`;
- choose an empty grid cell and enter the relations of the path algebra in the form `rel :`
.....

Example:



Click `LaTeX` at the bottom of the q.uiver page and copy the generated `tikzcd` source code.

Create a new `txt` file named `yourfile.txt` inside the `ARquiver` directory, paste the copied source code into it, save the file, and close it.

Draw the AR quiver

Open `run.ipynb` and run the following cells in order:

```
# Compute algebra data: filename.txt -> filename.log
%run source/compute_all.py
```

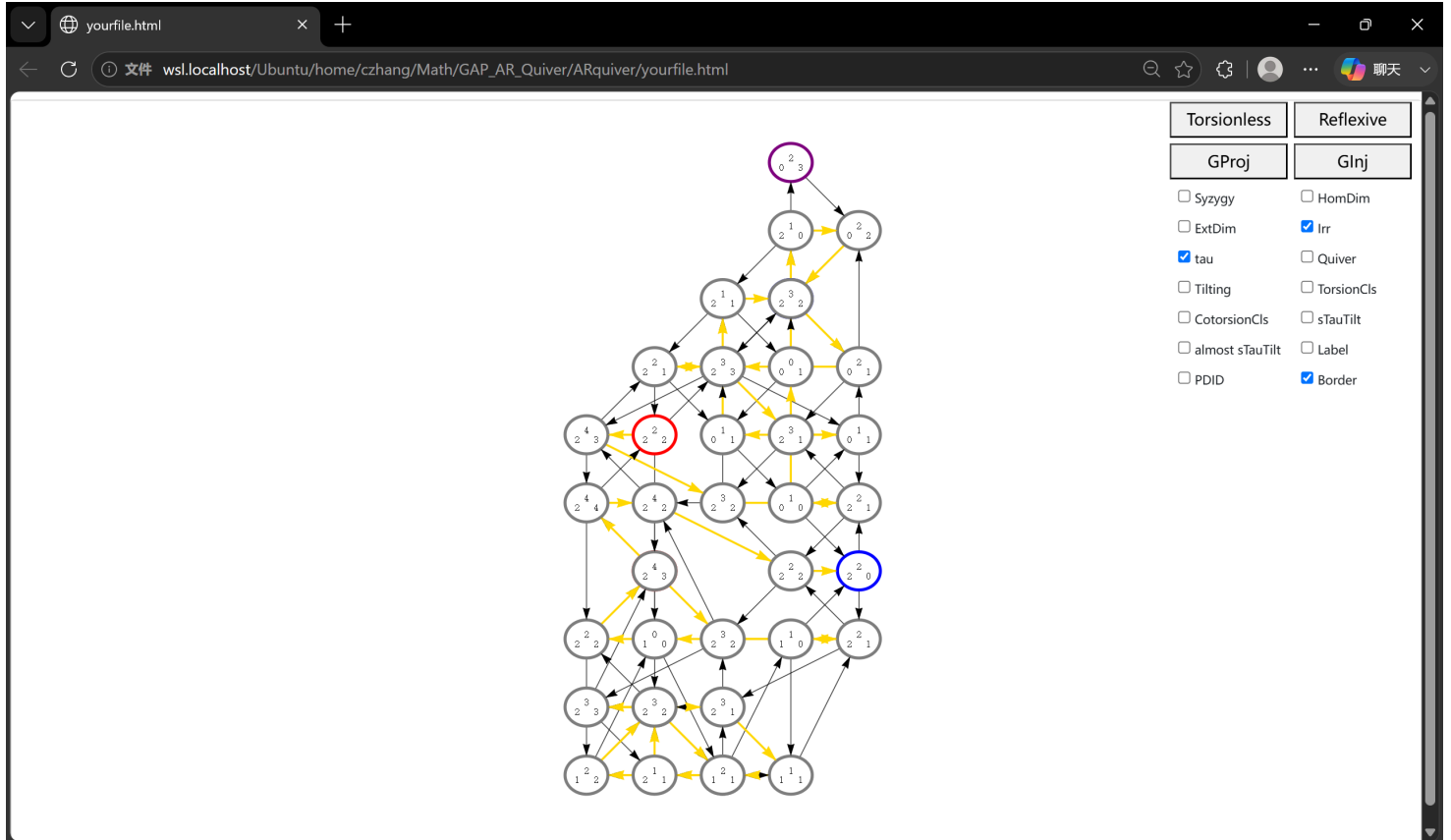
```
# Render interactive diagram: filename.log -> filename.html
%run source/render_all.py
```

The ARquiver directory will then contain:

- `yourfile.log`: the algebra computation log;
- `yourfile.html`: the interactive AR-quiver canvas.

Arrange the AR quiver into a standard form

Open `yourfile.html` to view the interactive visualization of the AR quiver.



We first describe the basic AR-quiver controls.

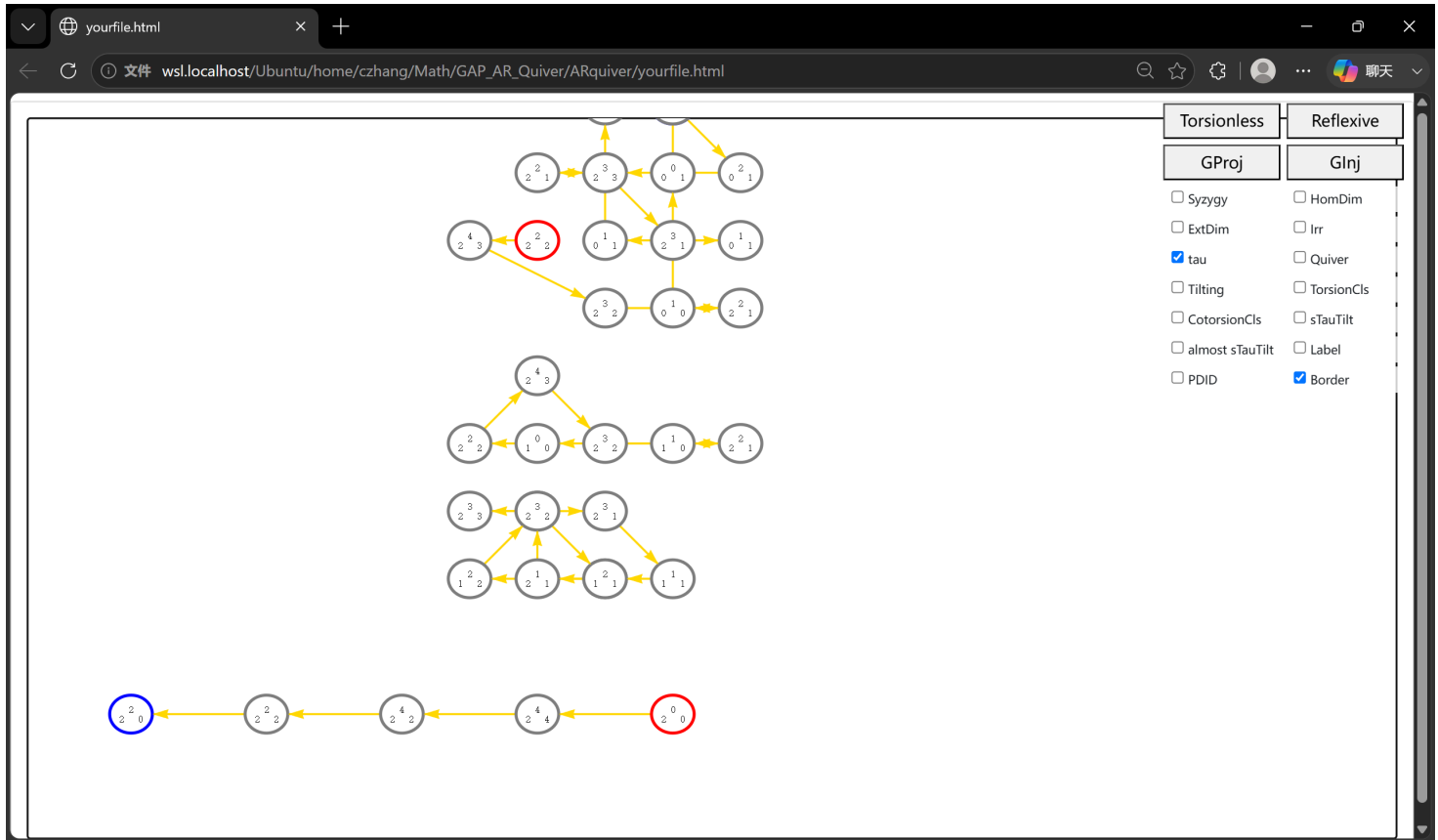
- Each ellipse represents an indecomposable module. Its dimension vector is arranged according to the vertex positions in the original `tikzcd` input.
- Purple vertices are both projective and injective. Red vertices are projective but not injective, while blue vertices are injective but not projective.
- `Irr`: show or hide irreducible morphisms, drawn as black arrows.
- `tau`: show or hide the AR translation $\tau = D\text{Tr}$, drawn as golden arrows.
- `Border`: show or hide vertex borders.
- `Ctrl + Z` undoes an operation, and `Ctrl + Y` redoes it.

The main manual task is to arrange the AR quiver into a readable standard form. Here are some useful guidelines.

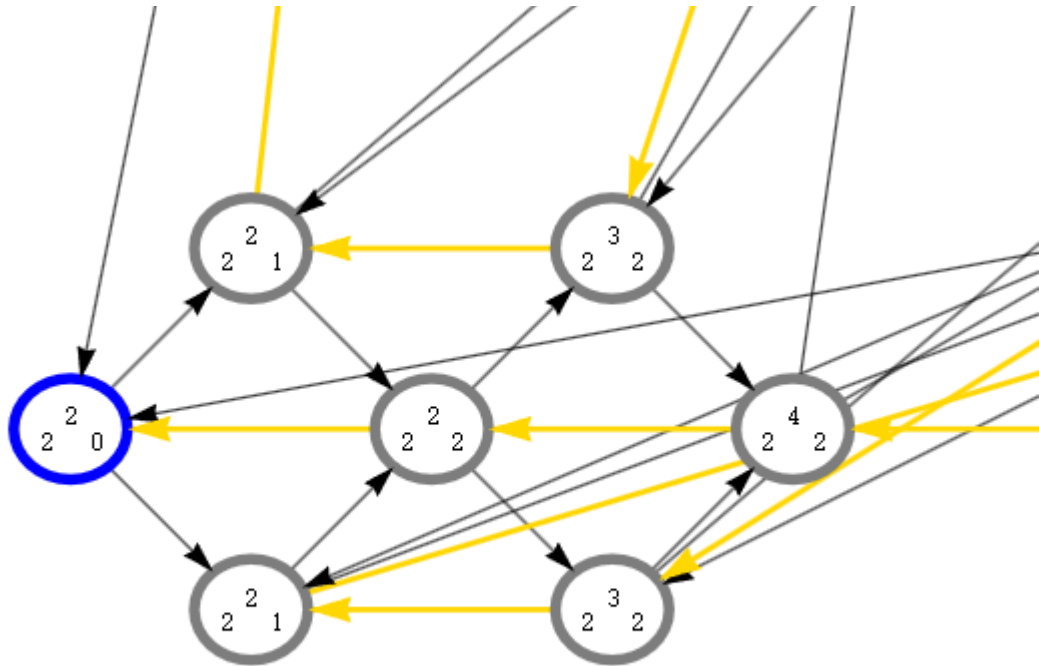
Lemma. The τ -orbits, namely the orbits of the golden arrows, are disjoint. Projective-injective objects do not belong to any τ -orbit. Every other indecomposable module belongs to exactly one τ -orbit. Hence each τ -orbit is one of the following two types:

- a straight path from an injective object to a projective object;
- a cycle that contains no projective or injective object.

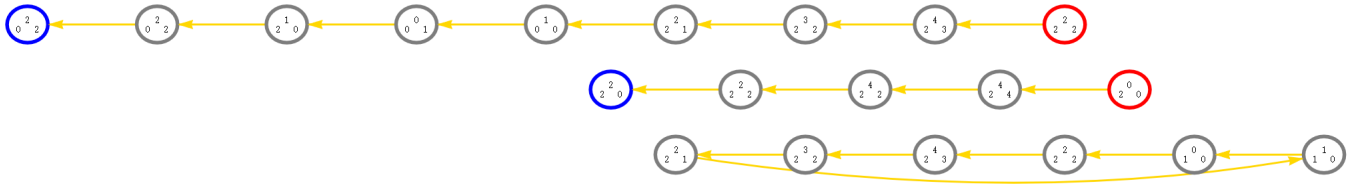
Following this lemma, we first arrange the projective object 2^2_0 . Turn off `Irr`, then place the arrow $2^2_0 \leftarrow 2^2_2$ horizontally in an empty region. Long-press the golden arrow to align the next arrow. This gives:



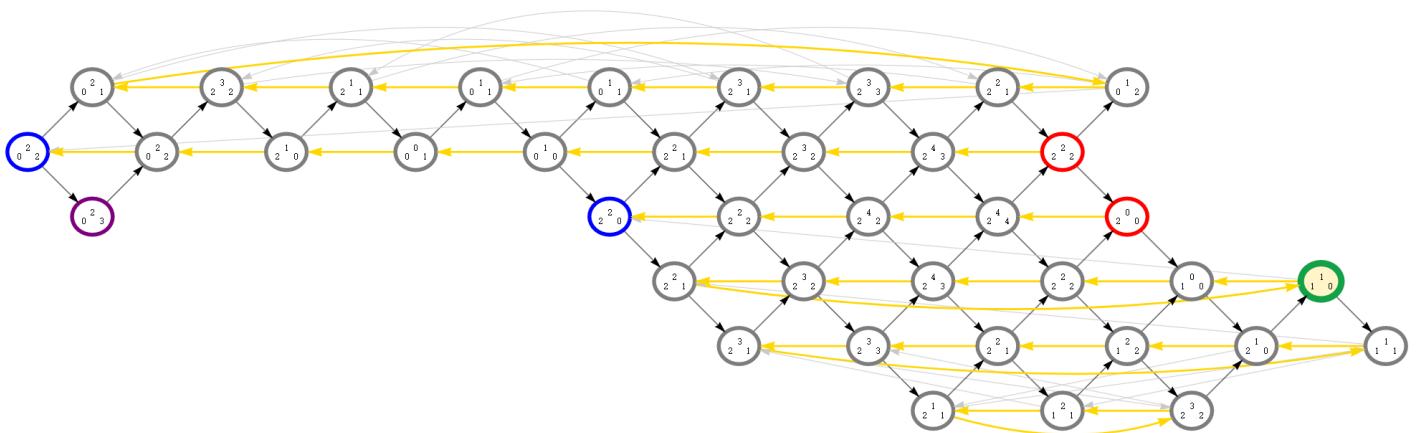
Next, turn on `Irr` and look for the almost split short exact sequence $2^2_0 \twoheadrightarrow \bigoplus M_i \twoheadrightarrow 2^2_2$.



Then turn off `irr` again and long-press the golden arrows for alignment. If a cycle appears, select an edge and use the `↑` and `↓` keys to adjust the curvature of the arrow. After a sequence of such operations, one obtains the τ -orbits:



Finally, turn off `tau` and adjust the curvature of the horizontal black arrows that were hidden behind the golden arrows. If some edges are visually inconvenient, double-click them to switch between dark and light colors. The final AR quiver is then obtained:



Features

The generated `filename.html` file is an interactive workspace for studying the Auslander--Reiten quiver and related homological data of the algebra defined by `filename.txt`. The page consists of a main canvas, a top navigation bar, and a sidebar opened by `Controls`. This introduction follows the order of the interface: first the canvas and top navigation, then each sidebar section from top to bottom.

Basic canvas interaction

Each ellipse on the canvas represents an indecomposable module. The label shown inside the ellipse is normally the dimension vector, arranged according to the shape of the original input quiver when this information is available. The internal node number is the identifier used throughout `filename.log`.

Basic operations:

- Drag a vertex to change the layout.
- Drag the background to pan the view.
- Use the mouse wheel to zoom.
- Hover over a vertex to see only its internal node number, for example 14.
- Double-click an edge to toggle between dark and light display colors.
- Select an edge and press $\uparrow$ or $\downarrow$ to adjust its curvature.

These operations only change the visualization. They do not recompute the algebra.

Top navigation bar

Controls

This opens or hides the sidebar. The sidebar contains all overlays, lists, and tools. It is divided into `View`, `Quivers`, `Modules`, `Classes`, `Tilting`, and `Tools`.

Fit graph

This recenters and rescales the current canvas so that the visible graph fits into the window. It is useful after dragging vertices or zooming far away from the main diagram.

Ctrl+Z and Ctrl+Y

These buttons duplicate the keyboard shortcuts.

- `Ctrl+Z` undoes recent layout or coloring operations.
- `Ctrl+Y` redoes an operation that was undone.

They are intended for manual arrangement of the AR quiver, especially when aligning τ -orbits.

Clear colors

This removes temporary colorings produced by the calculator, list selections, torsion/cotorsion classes, tilting objects, support τ -tilting objects, and module-class highlights. It does not delete arrows and does not change the underlying computation.

show dimension vector

This is the default label mode. Vertices display the dimension vectors of indecomposable modules. The source is the main `digraph Quiver` block in `filename.log`.

show label

This replaces each dimension vector by the internal node number. This is the best mode when comparing the picture with `filename.log`, since all tables in the log refer to modules by these numbers.

show custom label

This displays user-defined labels when available. If no custom label has been assigned, the vertex falls back to the dimension-vector label.

Ctrl+L hide/show UI

This keyboard shortcut hides or restores the user interface. It is useful for screenshots or for arranging a large graph without visual obstruction.

Sidebar: View

PD

Displays the projective dimension of each indecomposable module as a floating label. The value -1 denotes infinity or an unresolved value. The data come from

```
PDID := [[node, pd, id], ...];
```

ID

Displays injective dimension, using the third entry of the same `PDID` table.

Top

Displays the simple factors of the top of each indecomposable module. Multiplicities are preserved. The data source is

```
TopSoc := [[node, top, soc], ...];
```

Soc

Displays the simple factors of the socle of each indecomposable module, using the same `TopSoc` table.

Borders

Shows or hides vertex borders. The convention is:

- blue border: injective module;
- red border: projective module;
- purple border: projective–injective module;
- grey border: ordinary indecomposable module.

The source in the log is

```
Projective modules found (Node IDs): [...]
Injective modules found (Node IDs):  [...]
```

Sidebar: Quivers

This section overlays auxiliary quivers on top of the AR quiver.

AR irreducible arrows

Shows or hides the black irreducible morphisms of the AR quiver. These are the arrows in

```
digraph Quiver { ... }
```

If multiple identical arrows occur, the display may collapse them visually while keeping multiplicity information.

Translation quiver τ

Shows the AR translation quiver. A golden arrow

$$M \longrightarrow N$$

means $N = \tau M$. This is useful for arranging the AR quiver into τ -orbits. The data source is

```
digraph TranslationQuiver { ... }
```

Syzygy quiver

Shows the syzygy quiver. A pink arrow

$$X \longrightarrow Y$$

means that Y is an indecomposable direct summand of the kernel of a projective cover of X , i.e. Y is a summand of ΩX . The data source is

```
digraph SyzygySummand { ... }
```

The calculator uses this quiver to compute $\Omega^n(X)$ by iteration.

Cosyzygy quiver

Shows the cosyzygy quiver. A green arrow

$$X \longrightarrow Y$$

means that Y is an indecomposable direct summand of the cokernel of an injective envelope of X , i.e. Y is a summand of ΣX . The data source is

```
digraph CosyzygySummand { ... }
```

The calculator uses this quiver to compute $\Sigma^n(X)$.

Radical quiver

Shows the radical quiver. A cyan arrow

$$X \longrightarrow Y$$

means that Y is an indecomposable direct summand of $\text{Rad}(X)$. Repeated arrows encode multiplicities. The data source is

```
digraph RadicalSummand { ... }
```

The calculator uses this quiver to compute $\text{Rad}^n(X)$.

Coradical quiver

Shows the coradical quiver. A purple arrow

$$X \longrightarrow Y$$

means that Y is an indecomposable direct summand of $X/\text{Soc}(X)$. Repeated arrows encode multiplicities. The data source is

```
digraph CoradicalSummand { ... }
```

The calculator uses this quiver to compute $\text{Corad}^n(X)$.

Hom dimension quiver

Shows nonzero Hom dimensions. A brown arrow

$$M \longrightarrow N$$

with label k means $\dim \operatorname{Hom}(M, N) = k$. The source is

```
digraph HomDim { ... }
```

Ext dimension quiver

Shows nonzero Ext^1 dimensions. A red arrow

$$M \longrightarrow N$$

with label k means $\dim \operatorname{Ext}^1(M, N) = k$. The source is

```
digraph ExtDim { ... }
```

Together with the syzygy quiver, this data is used by the calculator to evaluate higher Ext dimensions.

Original quiver $\mathcal{Q}$

Opens a draggable window displaying the original quiver of the algebra and its relations. The source in the log is

```
digraph Q { ... }
rel := ...;
```

The `see filename.txt` button opens the original input text. The `Open in q.uiver` button opens the q.uiver URL stored in the first line of the input file.

Sidebar: Modules

These buttons highlight special classes of indecomposable modules. They do not alter the graph.

Torsionless

Highlights non-projective torsionless modules. A module is torsionless if it embeds into a projective module, equivalently if the canonical map $M \rightarrow DDM$ is injective. The display color is cyan. The source is

```
Torsionless modules found (Node IDs): [...]
```

Reflexive

Highlights non-projective reflexive modules. A module is reflexive if the canonical map $M \rightarrow DDM$ is an isomorphism. The display color is purple. The source is

```
Reflexive modules found (Node IDs): [...]
```

Gorenstein projective

Highlights non-projective Gorenstein projective modules in green. The source is

```
Gorenstein projective modules found (Node IDs): [...]
```

Gorenstein injective

Highlights non-injective Gorenstein injective modules in red. The source is

```
Gorenstein injective modules found (Node IDs): [...]
```

Sidebar: Classes**Torsion classes**

Opens a list of torsion pairs. Each row has the form

```
T := [...] | F := [...]
```

Display convention:

- red vertices show the torsion class $\mathbb{T}$;
- green vertices show the torsion-free class $\mathbb{F}$.

Mathematically, $\mathbb{T}$ is closed under quotients and extensions, while $\mathbb{F}$ is closed under submodules and extensions. For a tilting module L , the induced torsion pair satisfies

$$T = \text{gen}(L) = \text{Ker Ext}^1(L, -), \quad F = \text{Ker Hom}(L, -).$$

The list is sorted by the stored class data. The source is

```
# --- TorsionPairTable --- #
T := [...] | F := [...]
```

Cotorsion classes

Opens a list of cotorsion pairs. Each row has the form

```
L := [...] | R := [...] | Hereditary := true/false
```

Display convention:

- blue vertices show the left class L ;
- red vertices show the right class R ;
- if a module belongs to both sides, half-coloring is used.

The `Hereditary` flag records whether the cotorsion pair is hereditary in the computed data. The list is sorted by the stored left and right classes. The source is

```
# --- CotorsionPairTable --- #
L := [...] | R := [...] | Hereditary := ...
```

The screenshot shows a web application interface for analyzing cotorsion pairs. On the left, a quiver diagram is displayed with nodes labeled 1 through 31. Nodes 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31 are blue, indicating they belong to the left class L . Nodes 1, 2, 3, 4, 5, 6, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31 are red, indicating they belong to the right class R . Nodes 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31 are half-colored, indicating they belong to both classes.

On the right, there are two tables: "Torsionless" and "Reflexive". The "Torsionless" table has a header "GProj" and a body with checkboxes for "Syzygy", "ExtDim", "tau", "Tilting", "CotorsionCls", "almost sTauTilt", and "PDID". The "Reflexive" table has a header "GInj" and a body with checkboxes for "HomDim", "Irr", "Quiver", "TorsionCls", "sTauTilt", "Label", and "Border".

Below the tables, there is a list of cotorsion pairs. The list is sorted by the stored left and right classes. The list contains 6 entries, each with a left class L , a right class R , and a hereditary flag.

L	R (数量+字典序)	Hereditary
1. $L=[1, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40]$	$R=[6, 15, 16]$	hereditary
2. $L=[1, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 25, 26, 27, 28, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40]$	$R=[6, 9, 15, 16]$	non-hereditary
3. $L=[1, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 24, 25, 26, 27, 28, 29, 30, 32, 33, 34, 35, 36, 37, 38, 39, 40]$	$R=[6, 10, 15, 16]$	non-hereditary
4. $L=[1, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 19, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 34, 35, 36, 38, 40]$	$R=[6, 15, 16, 20]$	hereditary
5. $L=[1, 4, 5, 6, 7, 8, 9, 11, 14, 15, 16, 18, 20, 21, 22, 23, 25, 26, 29, 30, 32, 35, 36, 37, 38, 39]$	$R=[6, 15, 16, 26]$	non-hereditary
6. $L=[1, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 25, 26, 27, 28, 30, 32, 33, 34, 35, 36, 37, 38, 39, 40]$	$R=[6, 15, 16, 27]$	non-hereditary

Sidebar: Tilting

Tilting modules

Opens the list of classical tilting modules. A module T is tilting if

- $\text{pd } T \leq 1$;
- $\text{Ext}^{\geq 1}(T, T) = 0$;
- there is a short exact sequence $A \twoheadrightarrow T^0 \twoheadrightarrow T^1$ with $T^0, T^1 \in \text{add}(T)$.

Display convention:

- grey vertices are the indecomposable summands of the selected tilting module;
- red vertices are in the induced torsion class;
- green vertices are in the induced torsion-free class.

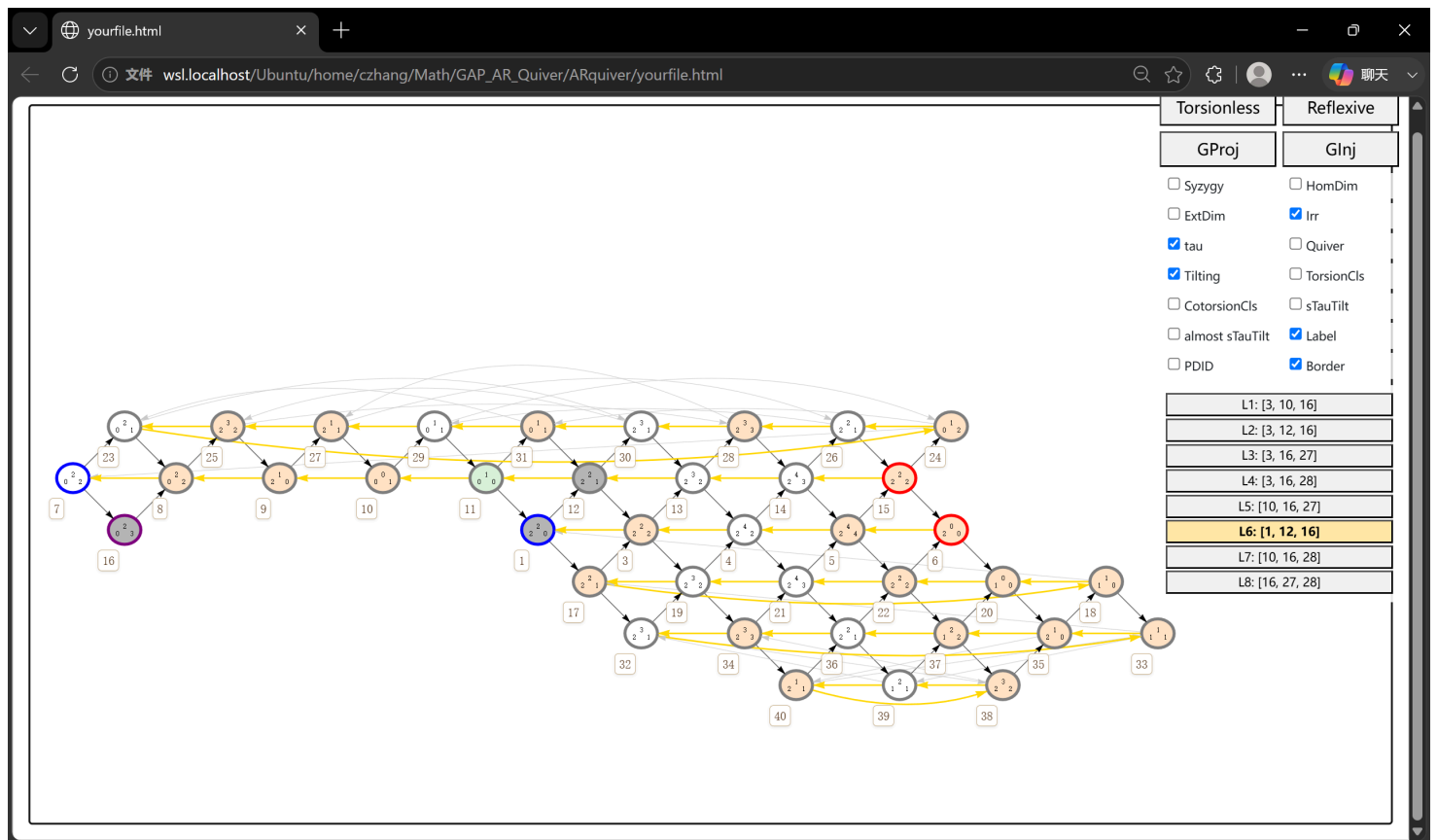
The source rows contain

```

L := [...]
F := [...]
T := [...]

```

where L is the tilting module, T is the torsion class, and F is the torsion-free class.



Support τ -tilting

Opens the list of support τ -tilting pairs. A pair (P, M) is support τ -tilting if P is projective, M is τ -rigid, $\text{Hom}(P, M) = 0$, and the total number of indecomposable summands of P and M is the number of vertices of the original quiver.

The source is

```

# --- SupportTauTiltingTable --- #
P := [...] | M := [...]

```

Almost support τ -tilting

Opens the list of almost support τ -tilting pairs. These have the same format as support τ -tilting pairs, but the total number of summands is one less than the number of vertices of the original quiver.

The source is

```
# --- AlmostSupportTauTiltingTable --- #
P := [...] | M := [...]
```

Sidebar: Tools

Calculator

The calculator performs operations using the data already stored in the HTML. Inputs and outputs use node label numbers.

Available operations:

- $\dim \operatorname{Ext}^k(A, B)$: computes total $\dim \operatorname{Ext}^k(A_i, B_j)$ over all selected labels.
- $\ker \operatorname{Ext}^k(A, -)$: returns all X with $\operatorname{Ext}^k(A_i, X) = 0$ for every selected A_i .
- $\ker \operatorname{Ext}^k(-, B)$: returns all X with $\operatorname{Ext}^k(X, B_j) = 0$ for every selected B_j .
- $\Omega^n(A)$: iterates the syzygy quiver.
- $\Sigma^n(A)$: iterates the cosyzygy quiver.
- $\operatorname{Rad}^n(A)$: iterates the radical quiver.
- $\operatorname{Corad}^n(A)$: iterates the coradical quiver.

Multiplicity convention: $3 + 5$ means $M_3 \oplus M_5$, while $3^2 + 5$ means $M_3^{\oplus 2} \oplus M_5$. Empty output is shown as $\emptyset$.

Run with GAP

Inside the calculator, this generates copyable GAP/QPA code reconstructing the quiver, algebra, projectives, injectives, simples, and serialized indecomposable modules when available. The script can also be downloaded as a `.g` file.

Export AR quiver to TeX

Exports the current arranged AR-quiver layout as TeX code. The export uses the current vertex positions.

Color legend

Opens a draggable legend explaining all visual conventions: vertex borders, module-class highlights, AR arrows, τ -arrows, syzygy, cosyzygy, radical, coradical, Hom/Ext arrows, torsion/cotorsion colors, tilting colors, support τ -tilting colors, calculator colors, and floating labels.

Acknowledgement

The author gratefully acknowledges the developers and maintainers of [GAP](#), [Binder](#), and [quiver](#) for providing essential tools and infrastructure used by this project. The implementation was inspired in part by A. Konovalov's [try-gap-in-jupyter](#) repository. The author also acknowledges the assistance of the AI model `chatgpt-5.5-thinking` during the development and refinement of the codebase.